



COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF ELECTRONICS

PROJECT REPORT Sobel Edge Detector

Submitted by

SURYADEV J

Under the guidance of

DR. TRIPTI S WARRIER

DR. NALESH S

design to deploy- short term Course

Contents

Abstract	2
1 Introduction	3
2 Objectives	4
3 Project Plan	5
4 Images and Report	7
RTL-to-GDSII Implementation	7
RTL Design	8
Coverage	9
Synthesis	9
Logic Equivalence Checking (LEC)	11
Post Synthesis Gate Level Simulation	12
Physical Design	13
Post Route Gate Level Simulation	20
SDF Delay Simulation	20
Static Timing Analysis	21
Timing Analysis Observation	22

Abstract

This project presents the design and implementation of a Sobel Edge Detection Processor using Verilog HDL and a complete RTL-to-GDSII design flow. Edge detection is a fundamental operation in digital image processing and computer vision applications, where the Sobel operator is widely employed for detecting object boundaries and intensity variations in images. The processor operates on a 3×3 pixel window and computes horizontal and vertical image gradients to generate an edge intensity output.

The functionality of the Sobel processor was verified through simulation using a SystemVerilog testbench, and functional coverage analysis was performed to ensure adequate verification of various image patterns and edge detection scenarios. The synthesized design was generated using Cadence Genus, followed by Logic Equivalence Checking (LEC) to verify functional equivalence between the RTL description and the synthesized gate-level netlist.

Physical implementation was carried out using Cadence Innovus, including floorplanning, placement, Clock Tree Synthesis (CTS), routing, and optimization. Static Timing Analysis (STA) was performed using Cadence Tempus to validate the timing requirements and ensure successful design signoff. The final design achieved successful timing closure, zero routing overflow, and satisfied all design rule constraints.

This project demonstrates the complete ASIC design methodology from RTL design to physical implementation and timing signoff using industry-standard EDA tools. The implemented Sobel Edge Detection Processor serves as a practical example of image processing hardware acceleration and provides valuable exposure to modern VLSI design and verification methodologies.

1. Introduction

The Sobel Edge Detection Processor is a hardware-based image processing system designed to detect edges in digital images by computing horizontal and vertical intensity gradients. In this project, a Sobel processor is implemented using Verilog HDL, where a 3×3 pixel window is processed to calculate the gradient components (G_x and G_y) and generate an edge intensity output. The hardware accelerator is designed to provide efficient and high-speed edge detection suitable for real-time image processing applications.

The project follows a complete ASIC design flow, starting from RTL design and functional verification, followed by synthesis, Logic Equivalence Checking (LEC), floorplanning, placement, Clock Tree Synthesis (CTS), routing, and Static Timing Analysis (STA). Industry-standard Cadence EDA tools are used throughout the implementation process.

The objective of this project is to demonstrate the design, verification, and physical implementation of an image processing hardware accelerator while achieving timing closure and generating a manufacturable GDSII layout through a complete RTL-to-GDSII implementation flow.

2. Objectives

- To design and implement a Sobel Edge Detection Processor using Verilog HDL.
- To develop a hardware accelerator capable of detecting edges in digital images by computing horizontal and vertical image gradients.
- To verify the functionality of the design through simulation and coverage analysis.
- To perform logic synthesis and generate an optimized gate-level netlist.
- To validate functional equivalence between the RTL and synthesized netlist using Logic Equivalence Checking (LEC).
- To implement the complete physical design flow, including floorplanning, placement, Clock Tree Synthesis (CTS), and routing.
- To perform Static Timing Analysis (STA) and achieve timing closure with positive setup and hold timing margins.
- To generate a final GDSII layout demonstrating successful RTL-to-GDSII implementation of the Sobel Edge Detection Processor.
- To gain practical exposure to industry-standard ASIC design methodologies and Cadence EDA tools used in modern VLSI implementation flows.

3. Project Plan

- Phase 1: Requirement Analysis
 - Define the functionality and specifications of the Sobel Edge Detection module.
- Phase 2: RTL Design
 - Develop the Verilog HDL code for the Sobel Edge Detection architecture.
- Phase 3: Functional Verification
 - Create a testbench and verify the design functionality through simulation and coverage analysis.
- Phase 4: Logic Synthesis
 - Synthesize the RTL using Cadence Genus and generate area, power, and timing reports.
- Phase 5: Logic Equivalence Checking (LEC)
 - Verify equivalence between the RTL and synthesized netlist using Conformal LEC.
- Phase 6: Physical Design
 - Perform floorplanning, power planning, placement, CTS, routing, and post-route optimization using Innovus.
- Phase 7: Static Timing Analysis
 - Perform setup and hold analysis using Cadence Tempus and achieve timing closure.
- Phase 8: Final Verification and GDSII Generation

- Perform post-route GLS with SDF annotation, validate the final design, and generate the GDSII layout and project report.

4. Images and Report

RTL-to-GDSII Implementation of Sobel Edge Detection Accelerator

This section presents the complete RTL-to-GDSII implementation of the Sobel Edge Detection Accelerator using Cadence EDA Tools. The design is initially developed at the Register Transfer Level (RTL) and verified through simulation. Subsequently, the complete ASIC design flow, including synthesis, floorplanning, power planning, placement, clock tree synthesis (CTS), routing, timing analysis, and physical verification, is performed to achieve a GDSII-ready implementation. Post-route timing analysis, Static Timing Analysis (STA), gate-level simulation, and Logical Equivalence Checking (LEC) are conducted to ensure functional correctness and timing closure.

RTL Design

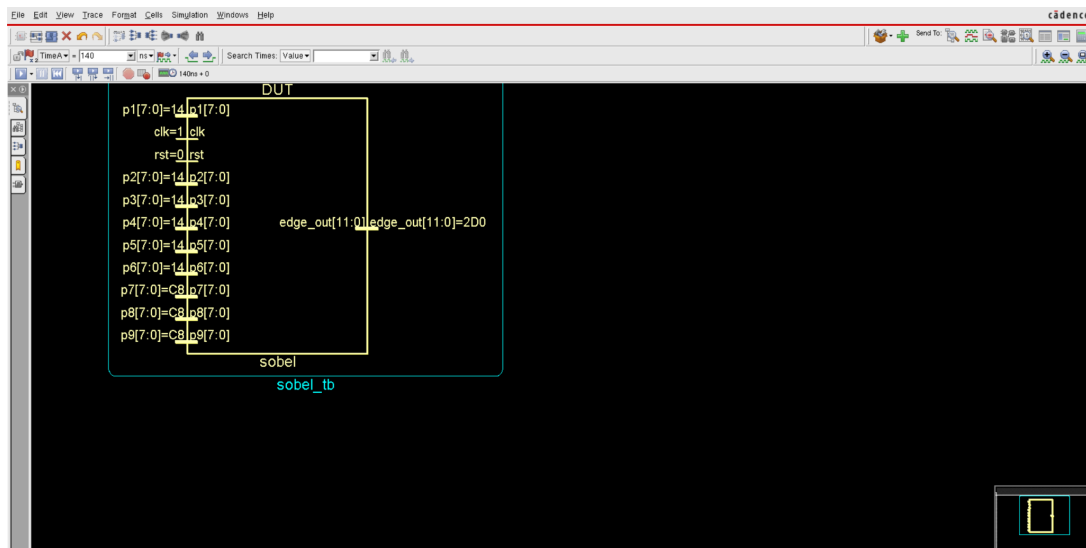


Figure 4.1: RTL Design – schematic

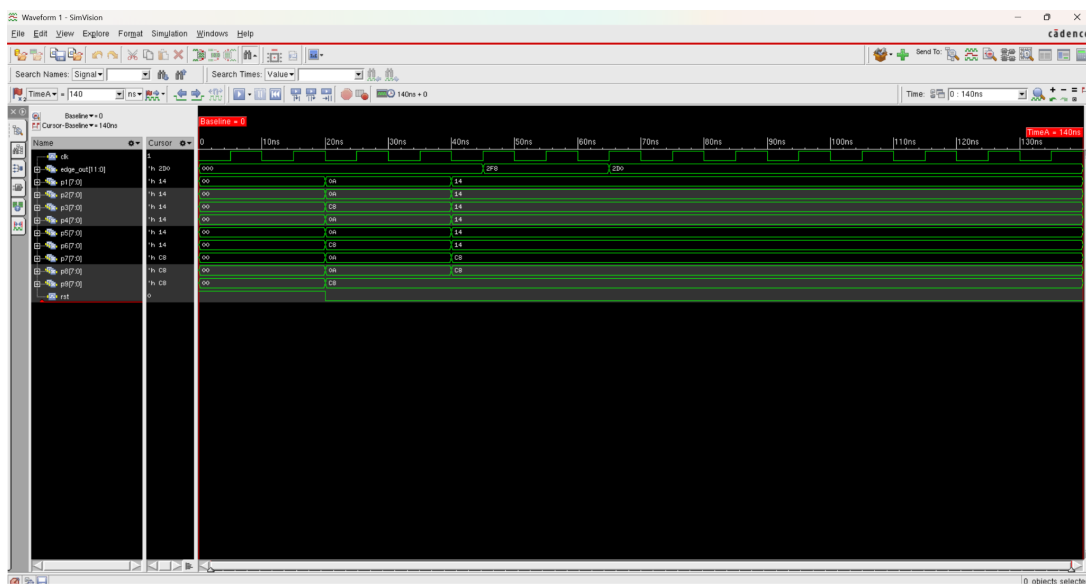


Figure 4.2: RTL Design – Simulation 2

Coverage

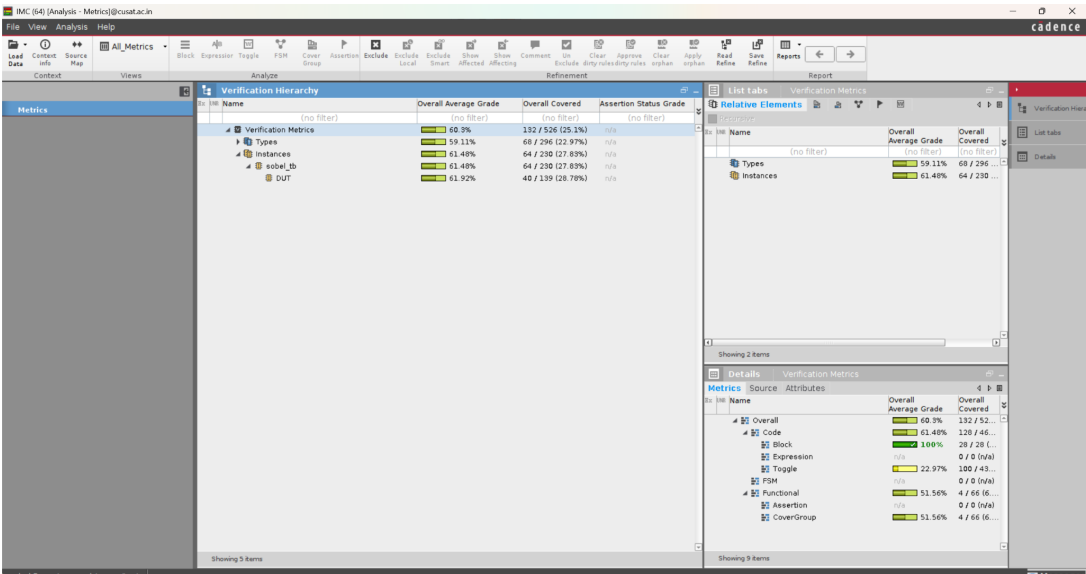


Figure 4.3: Coverage Report

Synthesis

Area

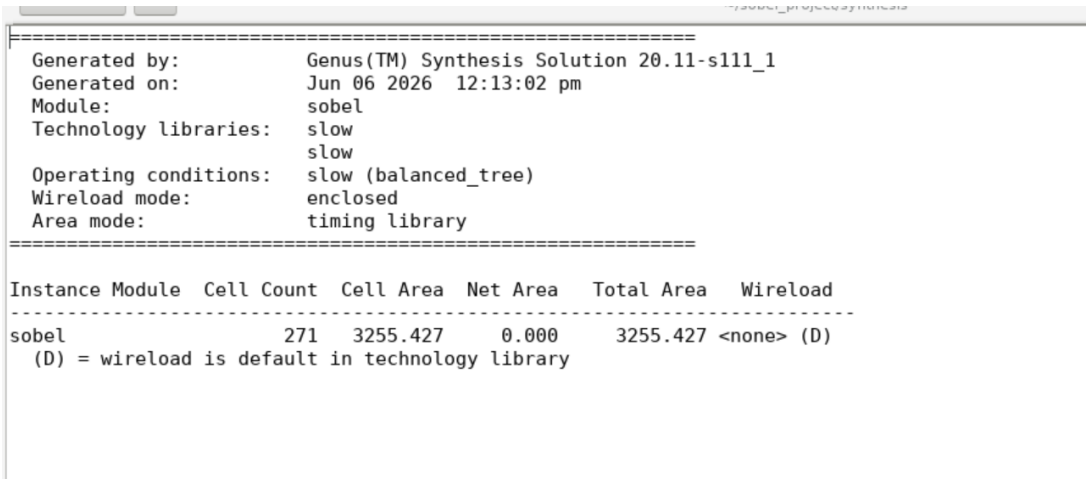


Figure 4.4: Synthesis – Area Report

Timing and Power

timing_4ns.rpt
~/sobel_project/synthesis

Generated by: Genus(TM) Synthesis Solution 20.11-s111_1
Generated on: Jun 06 2026 12:19:36 pm
Module: sobel
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode: timing library

Path 1: MET (191 ps) Setup Check with Pin gx_sl_reg[9]/CK->D
Group: clk
Startpoint: (F) p4[2]
Clock: (R) clk
Endpoint: (R) gx_sl_reg[9]/D
Clock: (R) clk

	Capture	Launch
Clock Edge:+	4000	0
Drv Adjust:+	0	0
Src Latency:+	0	0
Net Latency:+	0 (I)	0 (I)
Arrival:=-	4000	0

Setup:-	211
Uncertainty:-	100
Required Time:-	3689
Launch Clock:-	0
Input Delay:-	500
Data Path:-	2998
Slack:-	191

Exceptions/Constraints:
input_delay 500 constraints.sdc_line_14_29_1

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load	Trans	Delay	Arrival	Instance
#						(fF)	(ps)	(ps)	(ps)	(ps)	Location
#											
p4[2]		-	-	F	(arrival)	1	2.7	50	0	500	(-, -)

Figure 4.5: Synthesis – Timing Report

power_1ms.rpt
~/sobel_project/synthesis

Instance: /sobel
Power Unit: W
ODB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	6.86807e-06	4.59599e-04	4.55651e-05	5.12032e-04	70.07%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	8.46690e-06	1.27279e-04	6.39260e-05	1.99672e-04	27.32%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	1.90755e-05	1.90755e-05	2.61%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.53350e-05	5.86878e-04	1.28567e-04	7.30780e-04	100.00%
Percentage	2.10%	80.31%	17.59%	100.00%	100.00%

Figure 4.6: Synthesis – Power Report

LEC

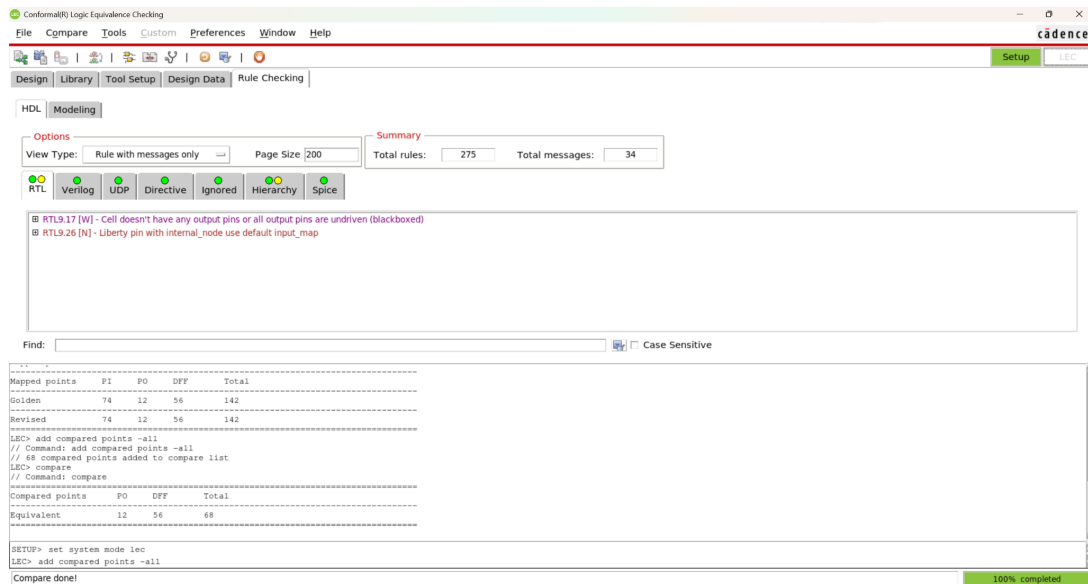


Figure 4.7: LEC – Result 1

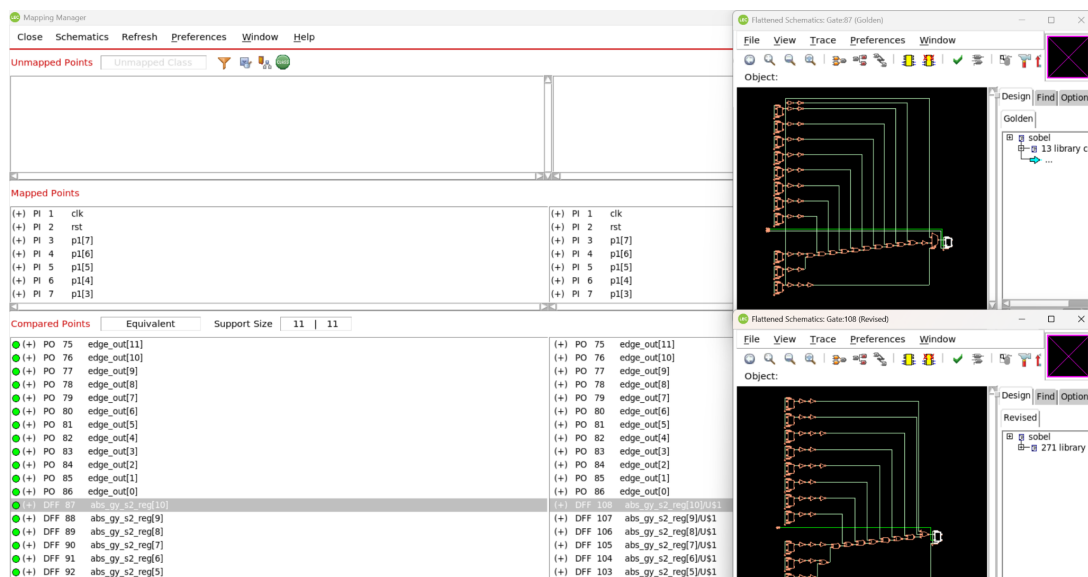


Figure 4.8: LEC – Result 2

Post Synthesis Gate Level Simulation

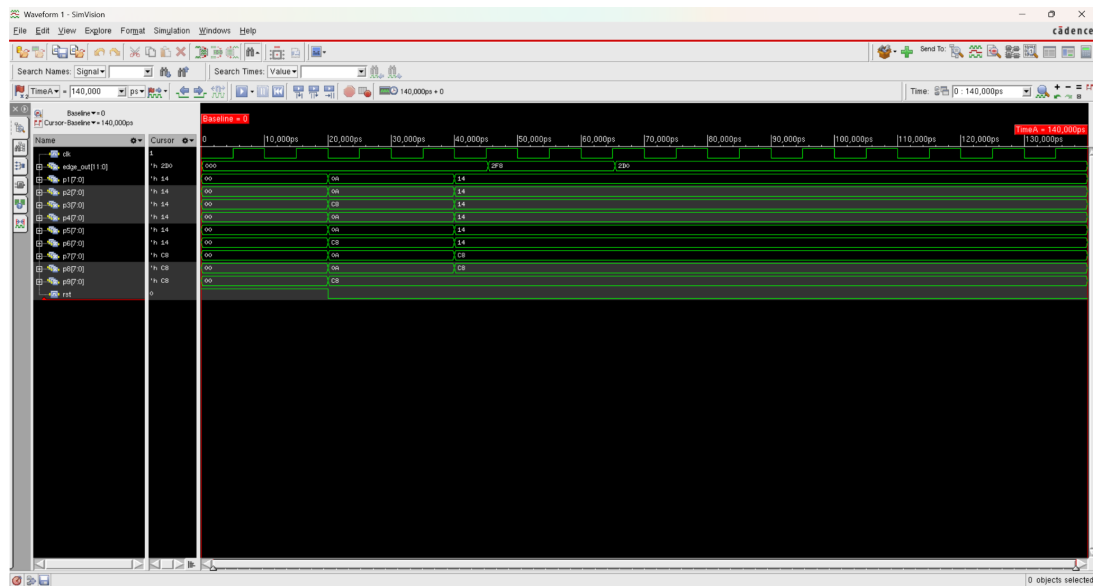
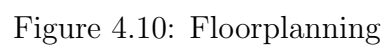


Figure 4.9: Post Synthesis Gate Level Simulation

Floorplanning



Pre-CTS

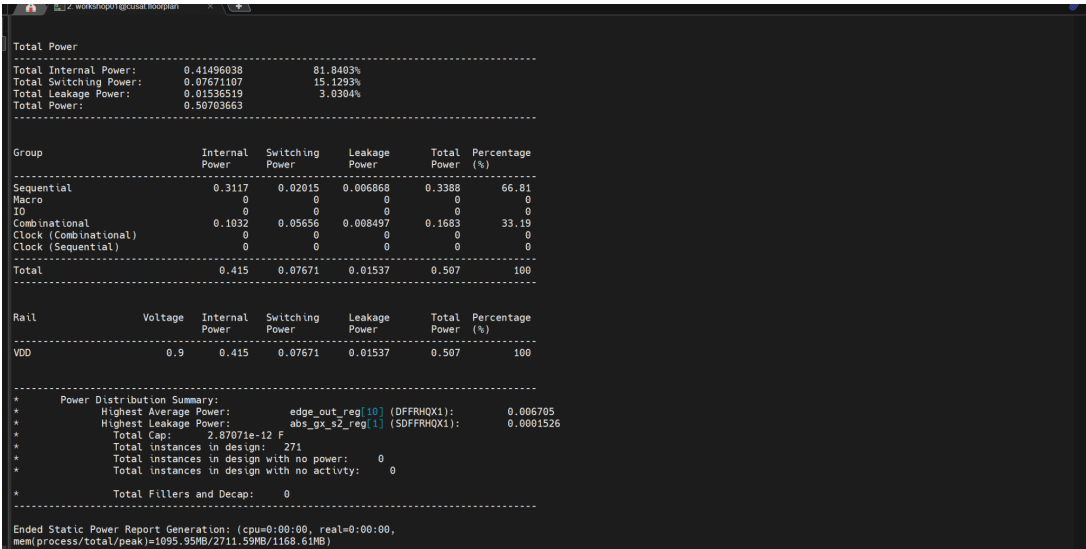


Figure 4.11: Pre-CTS – Result 1

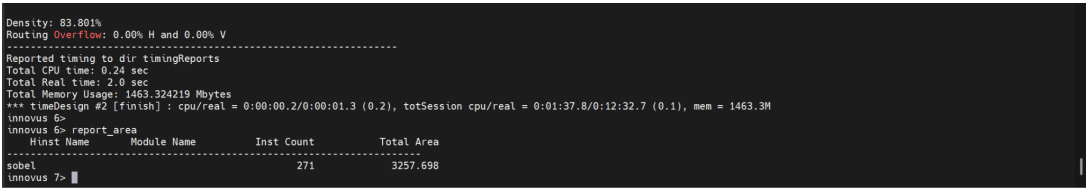


Figure 4.12: Pre-CTS – Result 2

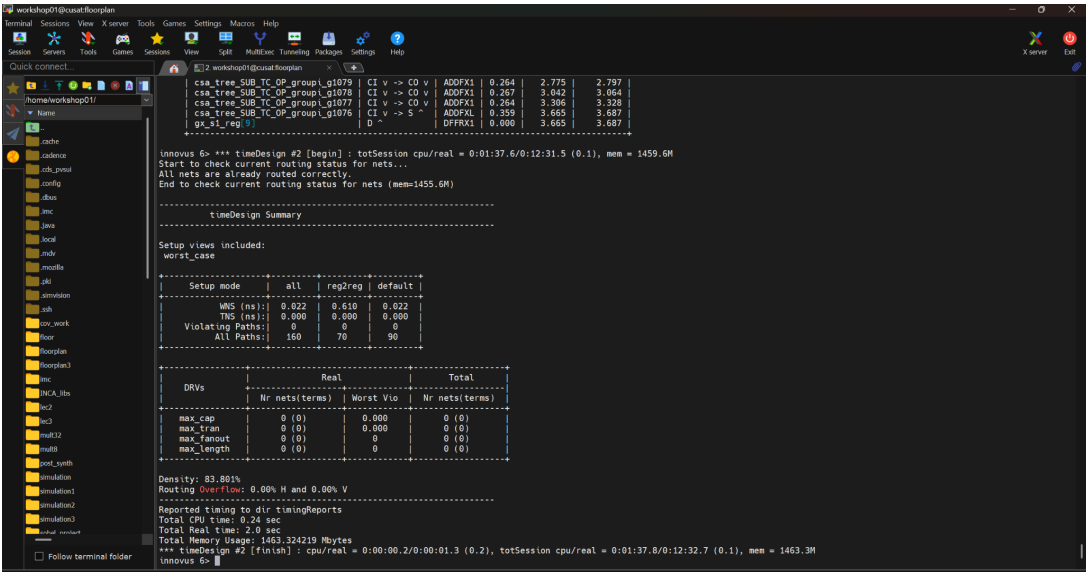


Figure 4.13: Pre-CTS – Result 3

Clock Tree Synthesis

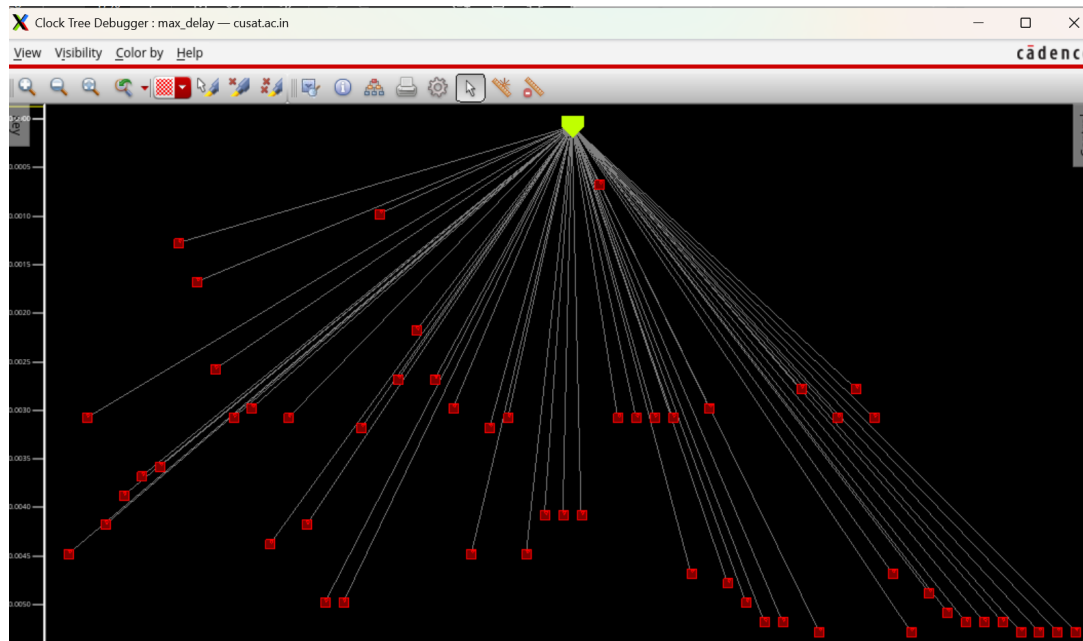


Figure 4.14: Clock Tree Synthesis

Post-CTS Report

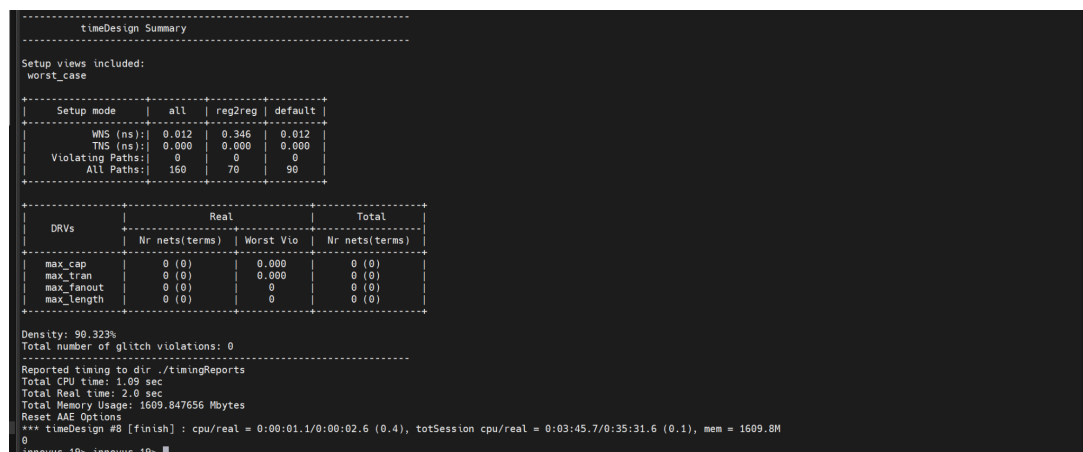


Figure 4.15: Post-CTS Report – Result 1

```

-----
timeDesign Summary
-----
Hold views included:
best_case

-----
| Hold mode | all | reg2reg | default |
-----
| WNS (ns): | 0.005 | 0.005 | 0.372 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 160 | 70 | 90 |
-----

Density: 90.323%

-----
Reported timing to dir ./timingReports
Total CPU time: 1.21 sec
Total Real time: 1.0 sec
Total Memory Usage: 1541.730469 Mbytes
Reset AAE Options
*** timeDesign #7 [finish] : cpu/real = 0:00:01.2/0:00:01.5 (0.8), totSession cpu/real = 0:03:37.6/0:33:47.4 (0.1), mem = 1541.7M
0

```

Figure 4.16: Post-CTS Report – Result 2

```

# Analysis Mode: MMCM Non-ULV
# Parasitics Mode: No SPEF/RCDB
# Signoff Settings: SJ Off
=====
Calculate delays in BcWc mode...
Start delay calculation (fullDC) (1 T). (MEM=1476.62)
Total number of fetched objects 421
AAE INFO: Total number of nets for which stage creation was skipped for all views 0
End delay calculation. (MEM=1490.3 CPU=0:00:00.1 REAL=0:00:00.0)
End delay calculation (fullDC). (MEM=1490.3 CPU=0:00:00.1 REAL=0:00:00.0)
*** Done Building Timing Graph (cpu=0:00:00.1 real=0:00:00.0 totSessionCpu=0:02:31 mem=1490.3M)

-----
timeDesign Summary
-----
Setup views included:
worst_case

-----
| Setup mode | all | reg2reg | default |
-----
| WNS (ns): | 0.022 | 0.610 | 0.022 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 160 | 70 | 90 |
-----

-----
| DRVs | | Real | | Total |
|-----|-----|-----|
| | | Nr nets(terms) | Worst Vio | Nr nets(terms) |
|-----|-----|-----|
| max_cap | 0 (0) | 0.000 | 0 (0) |
| max_tran | 0 (0) | 0.000 | 0 (0) |
| max_fanout | 0 (0) | 0 | 0 (0) |
| max_length | 0 (0) | 0 | 0 (0) |
|-----|-----|-----|

Density: 83.801%
Routing Overflow: 0.00% H and 0.00% V

-----
Reported timing to dir timingReports
Total CPU time: 0.42 sec
Total Real time: 1.0 sec
Total Memory Usage: 1459.566406 Mbytes
*** timeDesign #3 [finish] : cpu/real = 0:00:00.4/0:00:01.4 (0.3), totSession cpu/real = 0:02:31.6/0:25:29.3 (0.1), mem = 1459.6M
innovus >

```

Figure 4.17: Post-CTS Report – Result 3

Post-Route Report

```

-----
timeDesign Summary
-----
Hold views included:
best_case

-----
| Hold mode | all | reg2reg | default |
-----
| WNS (ns): | 0.005 | 0.005 | 0.372 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 160 | 70 | 90 |
-----

Density: 90.323%

-----
Reported timing to dir timingReports
Total CPU time: 1.11 sec
Total Real time: 1.0 sec
Total Memory Usage: 1556.199219 Mbytes
Reset AAE Options
*** timeDesign #10 [finish] : cpu/real = 0:00:01.1/0:00:01.4 (0.8), totSession cpu/real = 0:04:15.5/0:41:08.4 (0.1), mem = 1556.2M
innovus 21>

```

Figure 4.18: Post-Route Report – Result 1

```
-----
timeDesign Summary
-----

Setup views included:
worst_case

-----
| Setup mode | all | reg2reg | default |
-----
| WNS (ns): | 0.012 | 0.344 | 0.012 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 160 | 70 | 90 |
-----

-----
| DRVs | Real | Total | |
|---|---|---|---|
| | Nr nets(terms) | Worst Vio | Nr nets(terms) |
|-----|-----|-----|
| max_cap | 0 (0) | 0.000 | 0 (0) |
| max_tran | 0 (0) | 0.000 | 0 (0) |
| max_fanout | 0 (0) | 0 | 0 (0) |
| max_length | 0 (0) | 0 | 0 (0) |
|-----|-----|-----|

Density: 90.323%
Total number of glitch violations: 0
-----
Reported timing to dlr timingReports
Total CPU time: 1.17 sec
Total Real time: 3.0 sec
Total Memory Usage: 1619.917969 Mbytes
Reset AAE Options
*** timeDesign #9 [finish] : cpu/real = 0:00:01.2/0:00:02.6 (0.5), totSession cpu/real = 0:04:10.6/0:40:09.7 (0.1), mem = 1619.9M
innovus 21>
```

Figure 4.19: Post-Route Report – Result 2

Post-Route LEC

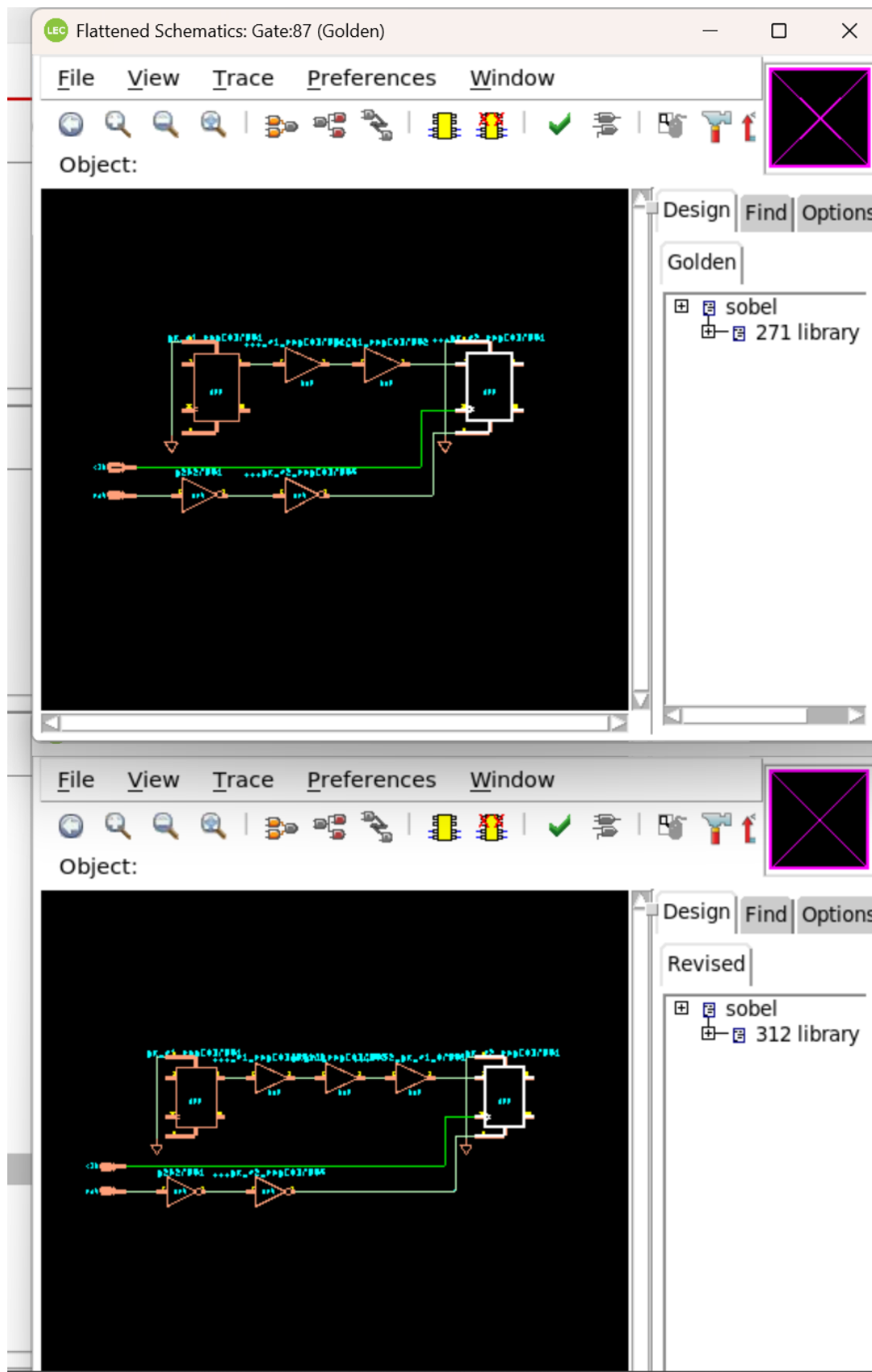


Figure 4.20: Post-Route LEC

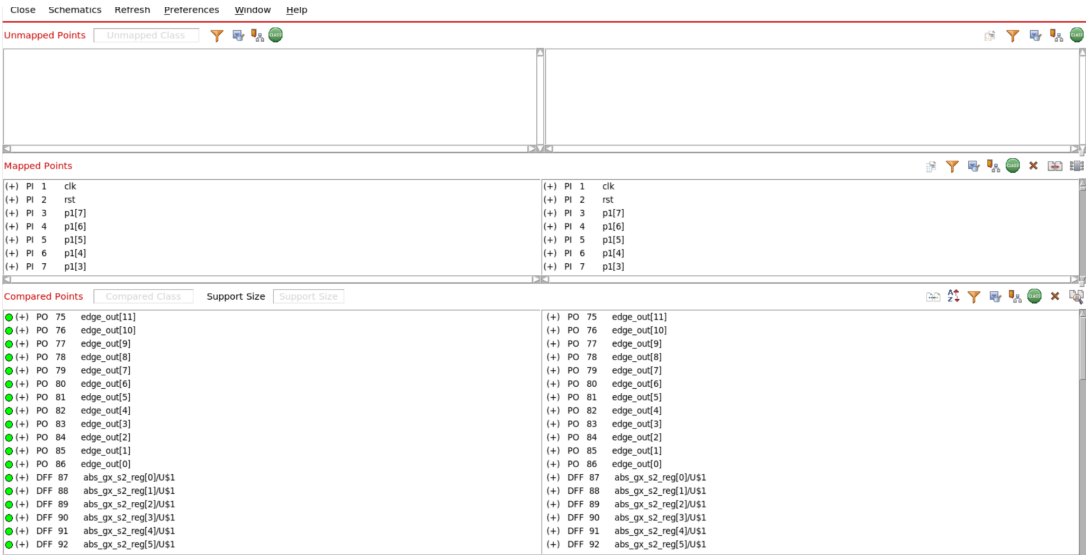


Figure 4.21: Post-Route LEC – Result 2

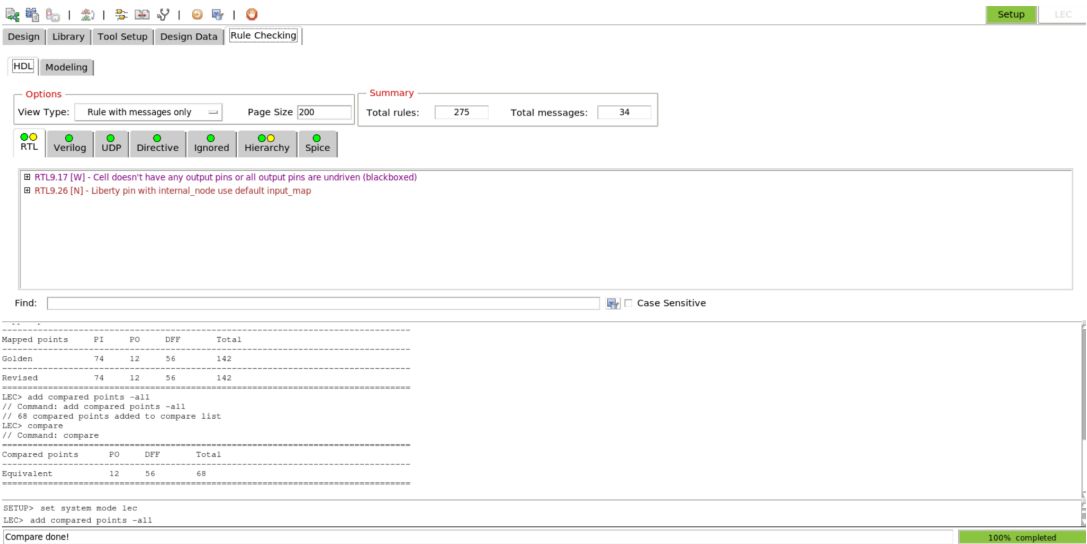


Figure 4.22: Post-Route LEC – Result 3

Post Route Gate Level Simulation

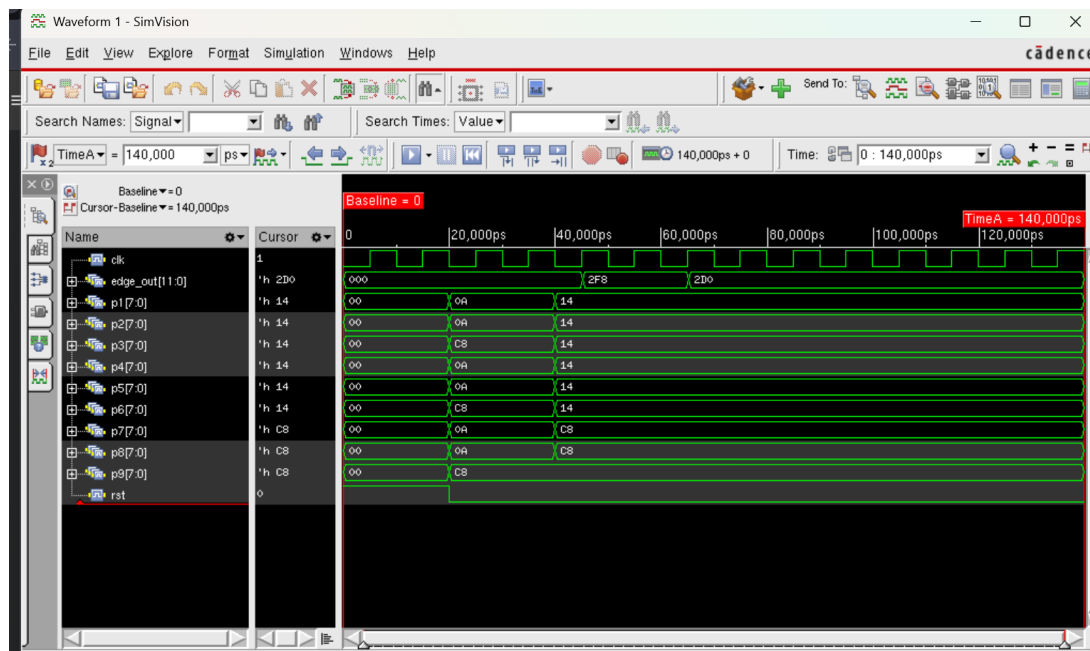


Figure 4.23: Post Route GLS

SDF Delay Simulation

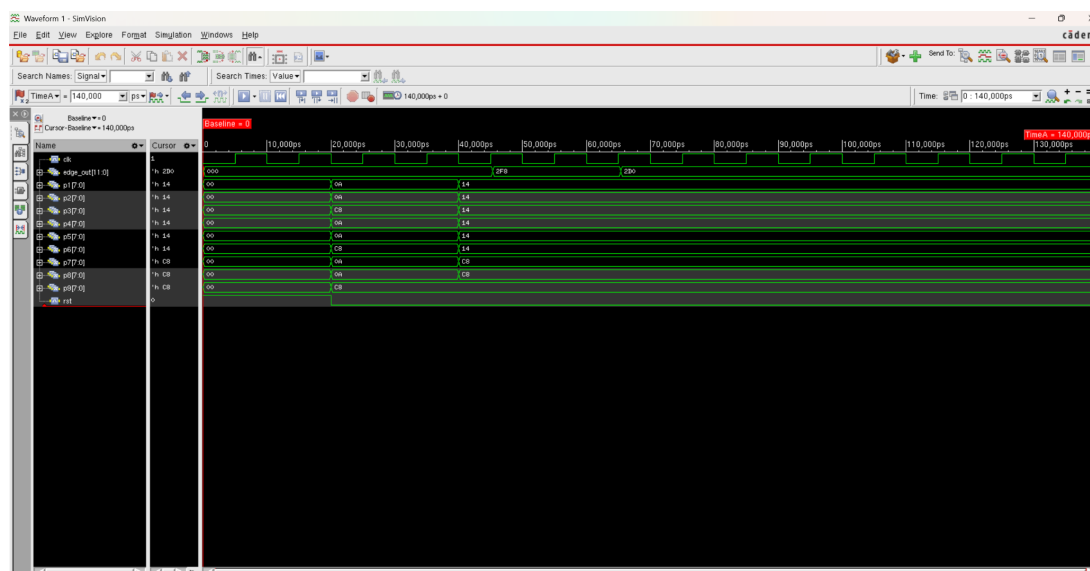


Figure 4.24: SDF Delay Simulation

Static Timing Analysis

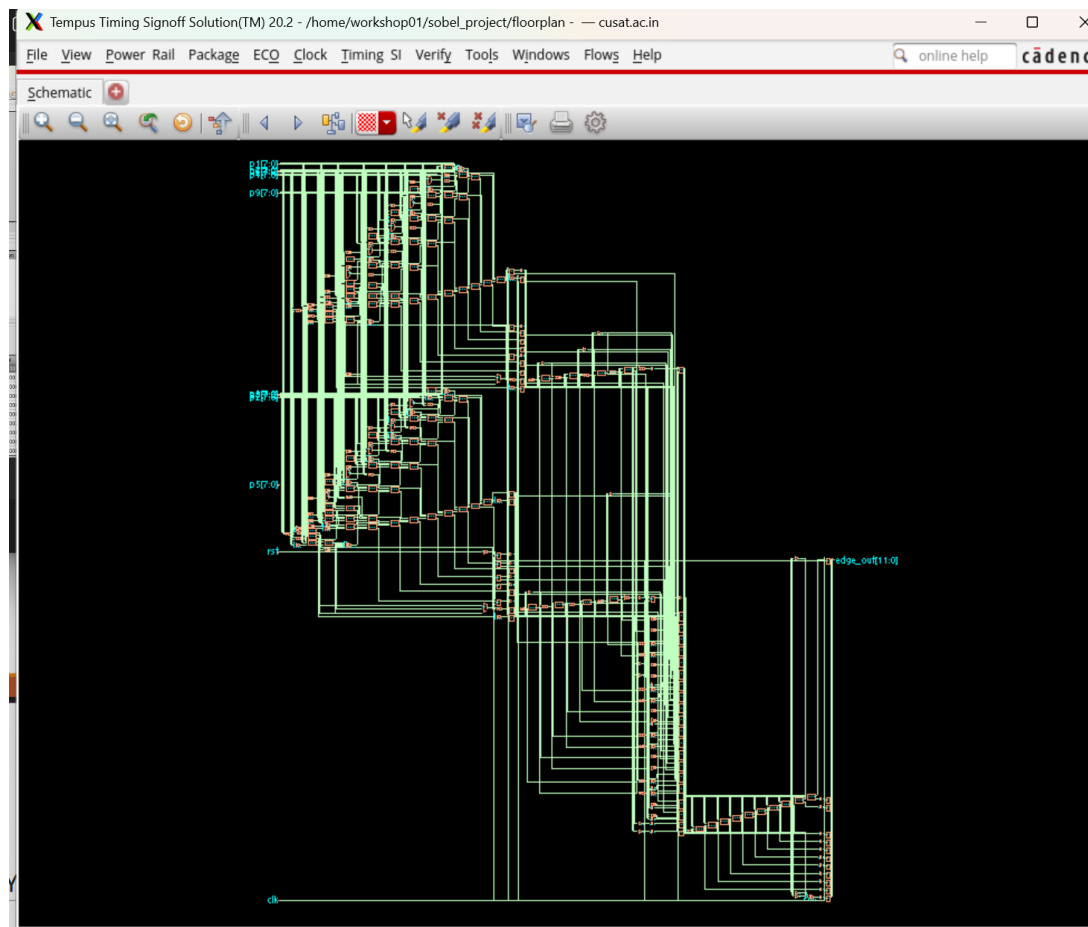


Figure 4.25: Static Timing Analysis – Result 1

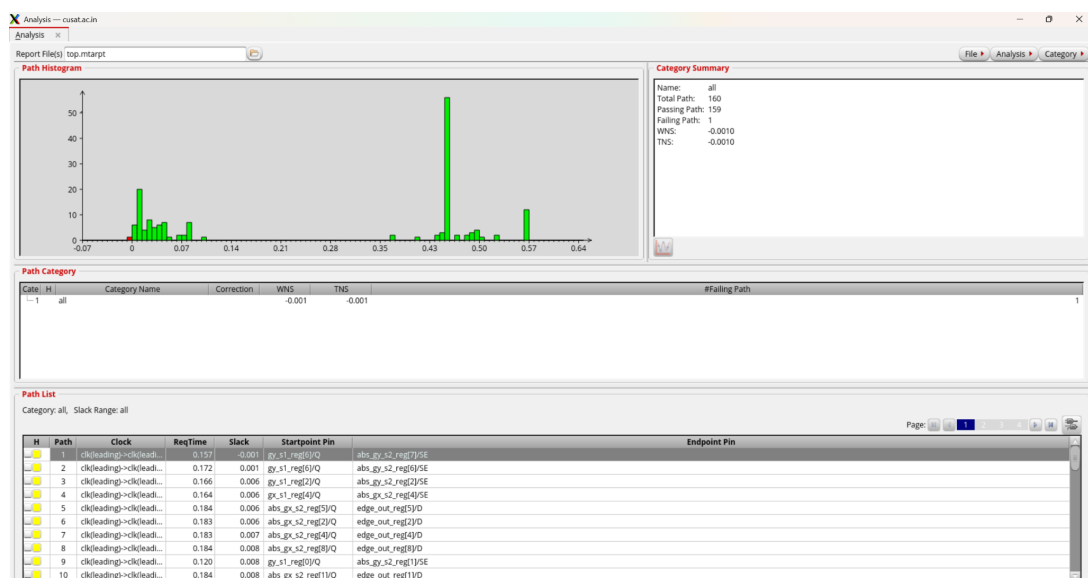


Figure 4.26: Static Timing Analysis – Result 2

Timing Analysis Observation

Post-route timing analysis was performed using both Cadence Innovus and Cadence Tempus. The Innovus timing reports indicated positive setup and hold slacks, suggesting that the design met the specified timing constraints during implementation. However, signoff analysis in Cadence Tempus reported a marginal hold violation with a Worst Negative Slack (WNS) of -0.001 ns (1 ps) in the best-case hold corner.

The discrepancy between Innovus and Tempus results can be attributed to differences in timing analysis methodologies, extraction accuracy, and signoff-level pessimism applied by Tempus. Since Tempus performs more accurate signoff timing analysis using detailed parasitic information and advanced variation modeling, the slight negative slack reported by Tempus is considered the final signoff result.

The observed violation is extremely small (1 ps) and can be regarded as a marginal hold issue arising from signoff pessimism rather than a significant functional timing failure. Nevertheless, in an industrial ASIC design flow, the Tempus signoff report is treated as the authoritative timing result, and such violations would typically require additional hold optimization before tape-out.